

Evolución del estado global: De Vuex a Pinia



Ambas herramientas centralizan el estado de la aplicación, pero Pinia es el gestor de estado recomendado actualmente para proyectos nuevos en Vue.

[Sintaxis simplificada]

Elimina la verbosidad de Vuex.

[TypeScript nativo]

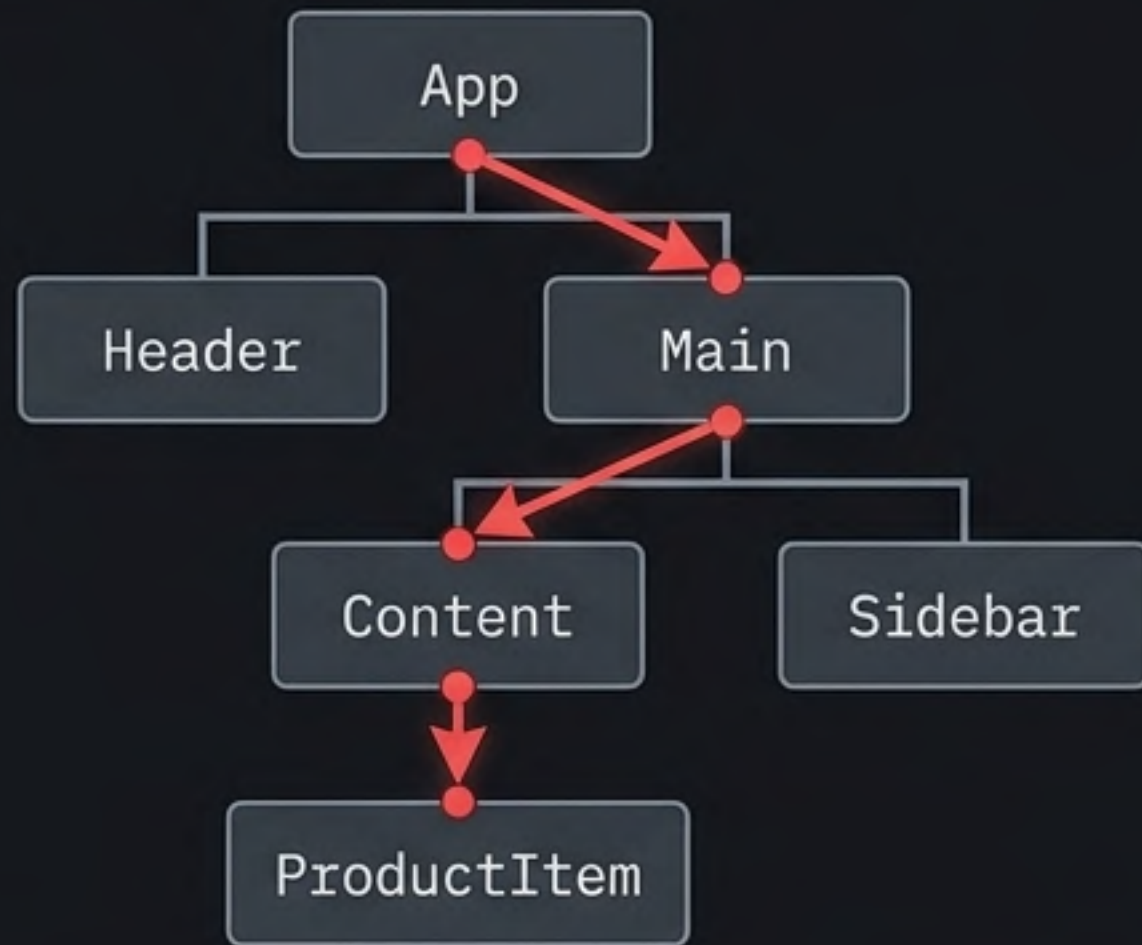
Inferencia automática de tipos.

[Diseño modular]

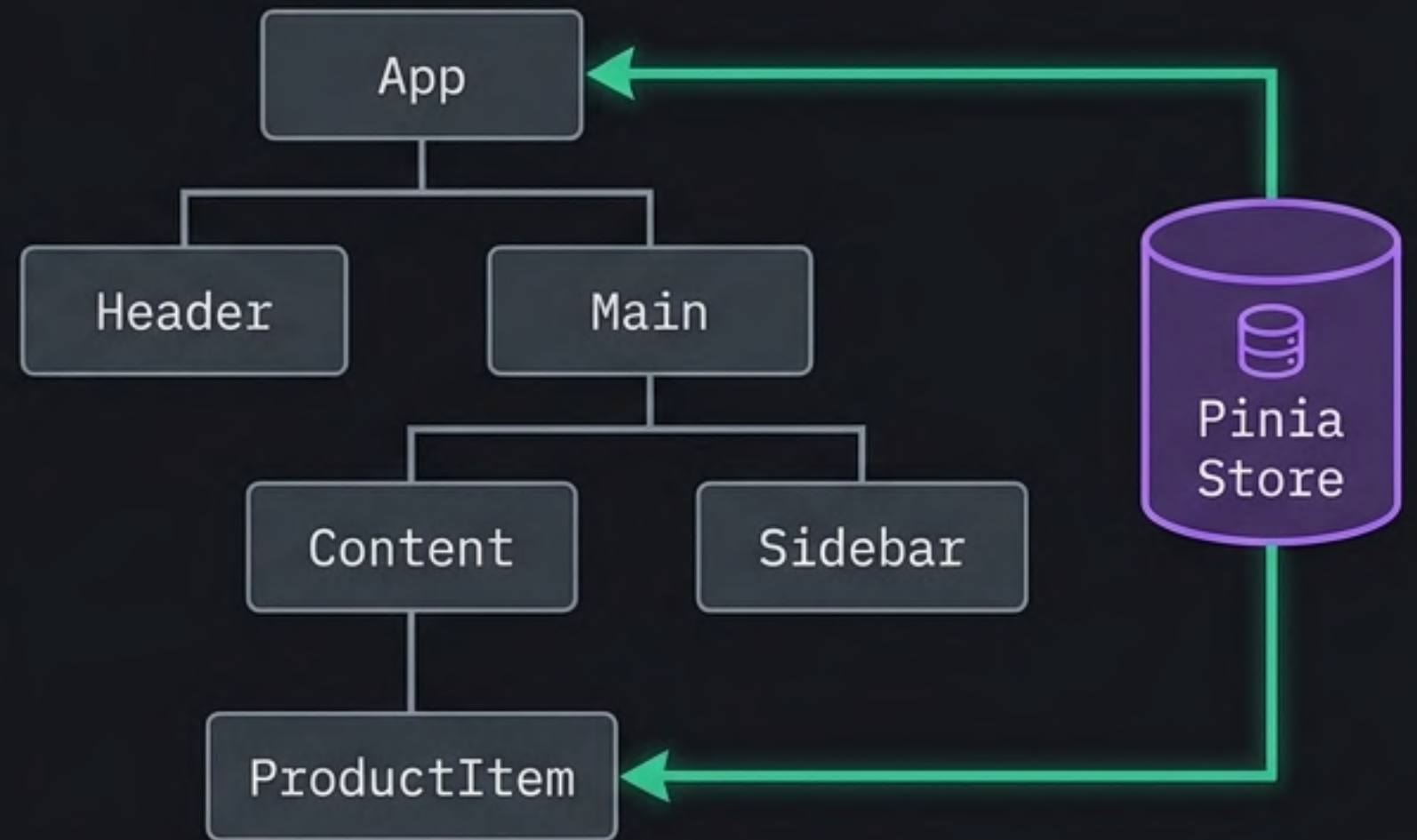
Adiós al árbol de estado único y anidado.

El problema raíz: Eliminando el Props Drilling

Props Drilling



Pinia Store



Evitamos pasar **props** a través de múltiples niveles, eliminamos eventos cruzados y mantenemos una **única fuente de verdad** sin datos duplicados.

Traducción directa: Vuex vs. Pinia

Vuex

state

getters

actions

mutations

modules

Pinia

state

getters

actions

~~Eliminado~~

No existen en Pinia

Stores separados

Diseño plano, no anidado

Los conceptos fundamentales se mantienen, pero la fricción desaparece.
El mayor cambio arquitectónico es la eliminación de las mutaciones.

Instalación y registro en el proyecto

```
pnpm add pinia
```

main.js

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
```

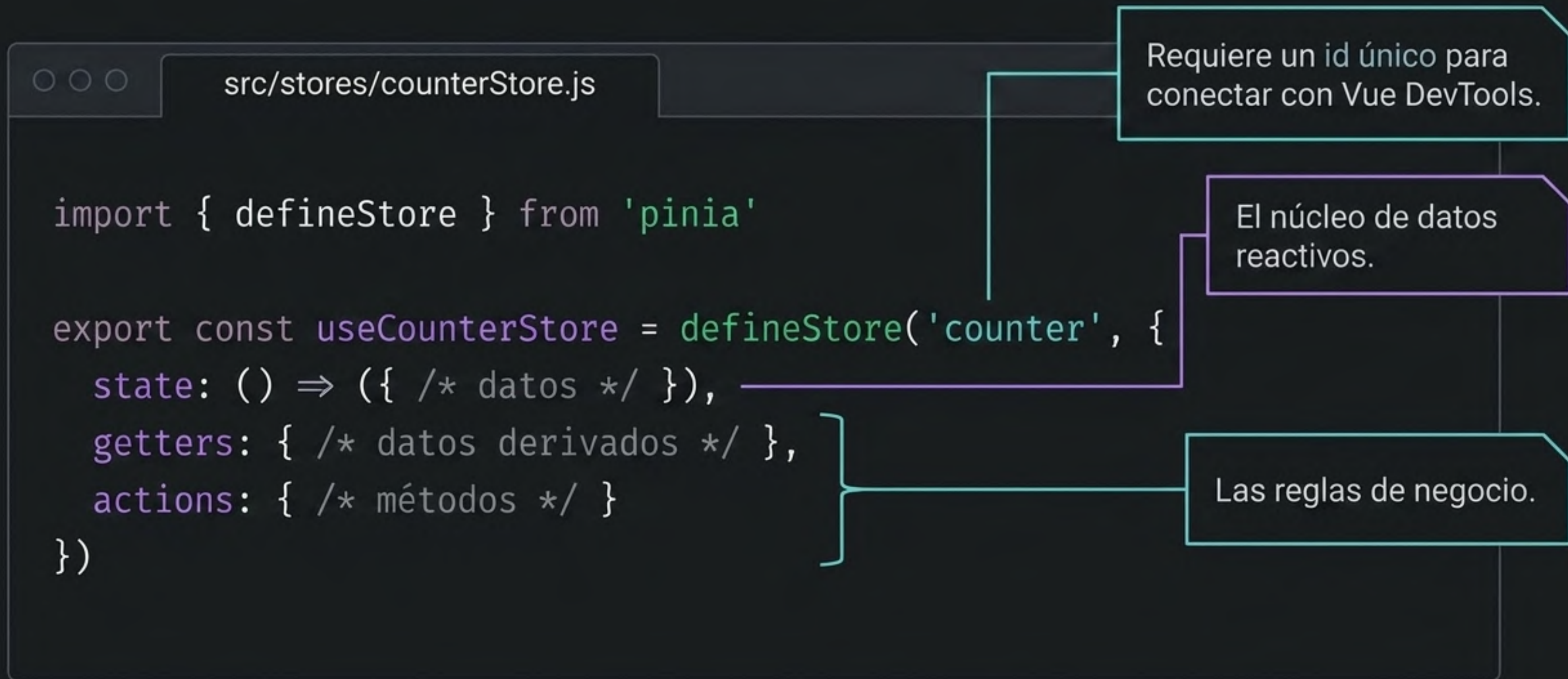
```
const pinia = createPinia()
const app = createApp(App)
```

```
app.use(pinia) // Se registra como plugin global
app.mount('#app')
```

Pinia se registra una vez en main.js y luego cualquier componente puede usar los stores.

Pinia se instala como dependencia estándar y se inyecta a nivel raíz antes del montaje.

La anatomía de un Store moderno



State: La base de datos reactiva global

src/stores/counterStore.js - state definition

```
state: () => ({  
  count: 0,  
  username: 'NovatoVue',  
  isLoggedIn: false,  
  loading: false,  
  error: null  
})
```

La Diferencia Clave en el Acceso



`store.state.modulo.valor`



`store.valor`

El estado se define como una función que retorna un objeto. En Pinia, el estado se aplanan y accedes a las propiedades directamente desde la instancia.

Consumo directo desde componentes

Componente.vue

```
<script setup>
import { useCounterStore } from '@stores/counterStore'

const counterStore = useCounterStore()
</script>

<template>
  <h2>Contador: {{ counterStore.count }}</h2>
</template>
```

Los stores en Pinia son inherentemente reactivos. Solo necesitas instanciar el store en el script y usarlo directamente en tu template usando Composition API.

Getters: Datos derivados sin mutación

src/stores/counterStore.js - getters

```
getters: {  
  double: (state) => state.count * 2,  
  greeting: (state) => `Hola, ${state.username}!`  
}
```



Equivalentes a las propiedades computed. Transforman o filtran el estado **sin modificarlo**. La inferencia de tipos (TypeScript) es completamente automática.

Actions: Lógica de negocio sin intermediarios

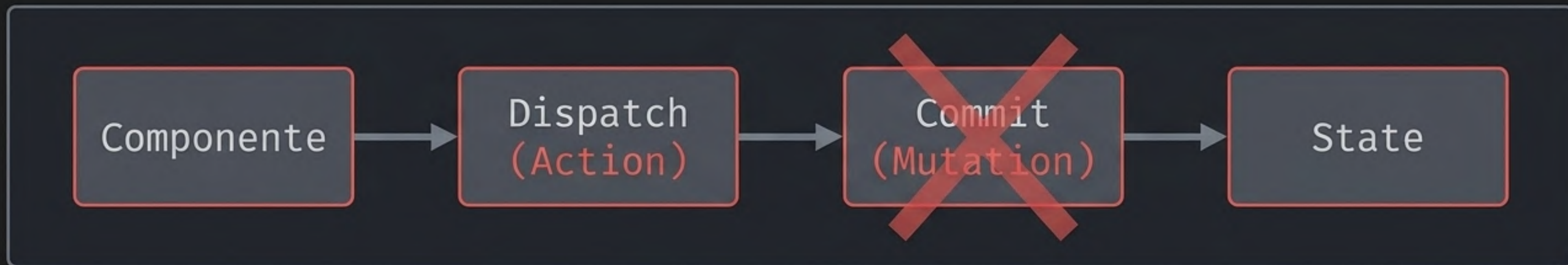
src/stores/counterStore.js - actions

```
actions: {  
  increment() {  
    this.count++  
  },  
  reset() {  
    this.count = 0  
  }  
}
```

Mutación directa con "this".
¡No se usa commit!

Equivalentes a los methods de un componente. Aquí ocurre la magia: accedemos y modificamos el estado directamente usando la palabra clave **this**.

La simplificación del flujo de datos



En Vuex, las `actions` manejaban asincronía y las `mutations` el cambio síncrono. En Pinia, una `action` hace ambas cosas, eliminando código verboso.

Caso Práctico: Store de Autenticación

●●● src/stores/useAuthStore.js

```
export const useAuthStore = defineStore('auth', {
  state: () => ({ isLoggedIn: false, username: '', error: null }),
  actions: {
    login(user, pass) {
      if (user === 'admin' && pass === '123') {
        this.isLoggedIn = true;
        this.username = user;
        this.error = null;
      } else {
        this.error = 'Credenciales inválidas';
      }
    },
    logout() { this.isLoggedIn = false; this.username = ''; }
  }
});
```

Toda la lógica de acceso, limpieza de variables y manejo de errores locales reside encapsulada de manera segura en la action correspondiente.

Consumo de APIs y estados asíncronos



api.js

```
const api = axios.create({ baseURL: 'https://api.ejemplo.com' })
```



usePostsStore.js - Action asíncrona

```
async fetchItems() {  
  this.loading = true;  
  this.error = null;  
  try {  
    const { data } = await api.get('/posts');  
    this.items = data;  
  } catch (err) {  
    this.error = err.message;  
  } finally {  
    this.loading = false;  
  }  
}
```

Las **actions** soportan asincronía nativamente. El uso de **try/catch/finally** es el estándar para gestionar el ciclo completo de la petición (**loading, error, datos**).

Renderizado de la UI asíncrona



PostsView.vue

```
<script setup>
import { onMounted } from 'vue'
import { usePostsStore } from '@stores/postsStore'

const postsStore = usePostsStore()
onMounted(() => postsStore.fetchItems())
</script>

<template>
  <div v-if='postsStore.loading'>Cargando...</div>
  <div v-else-if='postsStore.error'>{{ postsStore.error }}</div>
  <ul v-else>
    <li v-for='post in postsStore.items' :key='post.id'>
      {{ post.title }}
    </li>
  </ul>
</template>
```

Integración externa: Pinia y Vue Router

router.js

```
import { useAuthStore } from '@stores/authStore'

router.beforeEach((to, from) => {
  // Instanciación DENTRO del guard
  const authStore = useAuthStore()

  if (to.meta.requiresAuth && !authStore.isLoggedIn) {
    return '/login' // Redirigir si no hay sesión
  }
})
```

Instanciar aquí previene problemas en SSR

Para usar **Pinia** fuera de los componentes de Vue, invoca la función **useStore()** siempre dentro dentro de la **función o guard**, nunca a nivel raíz del archivo.

Checklist Arquitectónico: Reglas de Oro

✓ Estado compartido estricto

Usa Pinia solo para datos globales, no para variables locales de componentes.

✓ Getters para derivaciones

Transforma y filtra los datos del state usando siempre getters.

✓ Actions para todo

Concentra toda la lógica de negocio, cálculos y asincronía aquí.

✓ Adiós al commit

Modifica el estado directamente usando 'this'. No más mutaciones verbosas.

✓ Múltiples dominios

Evita un store gigante. Crea stores separados y planos (auth, posts, cart).

✓ Separación de responsabilidades

Las vistas llaman actions; las actions concentran lógica, API y cambios de estado.